RISC-201 ALU Technical Report & Benchmark Analysis

Course: CSC-201 Computer Organization & Architecture | **Author:** Arjun Phaneesh | **Component:** Arithmetic Logic Unit (ALU)

1. System Architectural Overview & Flag Rules

The RISC-201 ALU serves as the primary computational engine supporting 32-bit integer arithmetic, IEEE 754 single-precision soft-float operations, multi-radix multiplication, iterative division, and logical bitwise operations across general-purpose registers (R0–R15).

Status Flag Generation Rules: Status flags are evaluated dynamically at execution completion:

- **Zero (Z):** Set when the 32-bit execution result equals zero (`r.value == 0`).
- **Negative (N):** Set when the Most Significant Bit (MSB, bit 31) of the result is 1.
- **Carry (C):** Set on unsigned carry-out from Kogge-Stone addition/subtraction or bit shift-out.
- **Overflow (V):** Evaluated via `((a ^ res) & (b ^ res) & SIGN_BIT) != 0` for addition, and `((a ^ b) & (a ^ res) & SIGN_BIT) != 0` for subtraction.
- **Greater Than (G):** Evaluated as `(!Z) && (N == V)` for signed comparisons.

2. Core Sub-Module Algorithms & Implementation

32-bit Parallel Prefix Addition (Kogge-Stone): Achieves true $O(log_2 N)$ delay across 5 parallel prefix stages:

- *Pre-Processing:* Generate `G_i = A_i & B_i` and Propagate `P_i = A_i ^ B_i` signals are computed simultaneously, incorporating carry-in into bit 0.
- *Parallel Prefix Tree:* Across 5 iterations `(i = 1, 2, 4, 8, 16)`, prefix logic computes global generate and propagate signals via bitwise shifts: `G = G | (P & (G << i))` and `P = P & (P << i)`.
- *Post-Processing:* Final sum vector is calculated as `sum = P_orig ^ (G << 1) ^ carry_in`.

Multi-Radix Multiplication (Radix-4 Booth): Scans a 3-bit sliding window `{y_{i+1}, y_i, y_{i-1}}` to recode operands into digits in *{-2, -1, 0, +1, +2}*, reducing partial product accumulation steps from 32 down to 16 iterations.

IEEE 754 Soft-Float & Newton-Raphson Division: Implements IEEE 754 single-precision routines without hardware FPU instructions. Division employs Newton-Raphson iteration in Q30 fixed-point arithmetic via `reciprocalNR`, normalizing divisor *D* into [0.5, 1.0) and executing 5 refinement steps `x_{k+1} = x_k * (2 - D * x_k)`.

3. Empirical Performance Benchmarking & Analysis

Micro-benchmarks measured execution latency in nanoseconds per operation (ns/op) taking the minimum of 15 trials over a 100×1000 batch.

Algorithm	8-bit	16-bit	32-bit	48-bit
ADD (RCA)	35.48	60.03	108.65	167.19
ADD (CLA)	34.37	57.54	101.44	148.60
ADD (KS)	19.00	20.23	18.61	20.46
MUL (Booth2)	33.61	55.73	99.88	146.10
MUL (Booth4)	32.39	53.00	90.92	132.90
DIV (Restor)	49.81	85.88	154.42	229.50

Algorithm	8-bit	16-bit	32-bit	48-bit
DIV (NonRes)	30.90	76.94	115.23	156.75
DIV (Goldsch)	32.56	50.03	59.04	66.99
DIV (NR)	28.13	42.87	50.45	54.84

4. Visual Performance Analysis

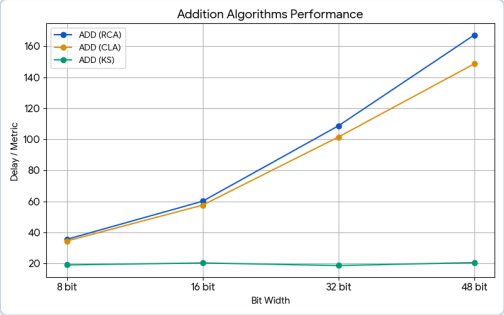


Figure 1: Addition (RCA, CLA, KS)

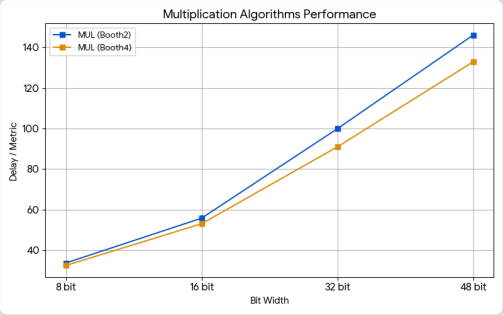


Figure 2: Multiplication (Booth2, Booth4)

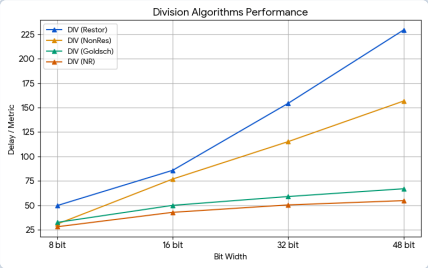


Figure 3: Division Algorithms Performance (Restoring, Non-Restoring, Goldschmidt, Newton-Raphson)